

Technical Documentation — Employee Management System

This document presents the implementation checklist and technical details of the Employee Management System, based on the supplied project structure, `server.ts`, and `types.ts`.

✓ Application starts without errors

Project structure:

- `src/` contains the frontend application, including `components/` and `services/`.
- `server.ts` implements the Express backend; `data/` stores employee data; `postman/` is provided for API testing.
- The project also contains `App.tsx`, `main.tsx`, `index.css`, `types.ts`, `package.json`, `vite.config.ts`, and `README.md`.

code

```
const app = express();
const PORT = 3000;
app.use(express.json());
app.listen(PORT, "0.0.0.0", ...);
```

The server uses Express JSON parsing and listens on port 3000. In development, Vite middleware serves the single-page frontend; in production, the built `dist/` folder is served.

✓ Employee data storage works correctly

- The Employee model contains ID, Employee ID, name, email, phone, department, designation, salary, joining date, address, status, and timestamps.
- The backend uses `data/employees.json` as persistent storage. The data directory/file is created when required.
- Create, update, and delete operations save the complete employee array back to the JSON file.

code

```
const DATA_DIR = path.join(process.cwd(), "data");
const DATA_FILE = path.join(DATA_DIR, "employees.json");
fs.writeFileSync(DATA_FILE, JSON.stringify(employees, null, 2));
```

✓ Dashboard summary is supported

- The dashboard endpoint calculates total, active, inactive, department-wise counts, and the five most recently joined employees.

code

```
GET /api/dashboard/summary
Response: { total, active, inactive, departmentSummary, recent }
```

✓ Create operation works

- A valid employee request is accepted through `POST /api/employees`.
- The backend validates Employee ID, names, email, phone, department, designation, salary, and joining date.
- A unique internal ID and timestamps are generated; the new employee is saved to the JSON data file and returned.

code

```
POST /api/employees
Success: 201 Created
Response: newly created Employee object
```

✓ Read/list operation works

- `GET /api/employees` returns the employee collection and supports search and filters.
- `GET /api/employees/:id` returns one employee by numeric internal ID or Employee ID.

code

```
GET /api/employees
GET /api/employees/:id
```

✓ Update operation works

- Both PUT and PATCH use the same update handler. Existing values are preserved when fields are not supplied.
- The backend validates changed fields, updates `updated_at`, persists the record, and returns the updated object.

code

```
PUT /api/employees/:id
PATCH /api/employees/:id
Success: 200 OK
```

✓ Delete operation works

- `DELETE /api/employees/:id` removes an employee by internal ID and persists the remaining records.
- A missing ID returns `404 Not Found`; successful deletion returns a message and deleted ID.

code

```
DELETE /api/employees/:id
Success: 200 OK
```

✓ CRUD operation summary

- **Create:** POST → validate → generate ID → save → return 201.
- **Read:** GET → load JSON → search/filter when requested → return records.
- **Update:** PUT/PATCH → find ID → validate → merge → save → return record.
- **Delete:** DELETE → find ID → remove → save → return confirmation.

✓ Validation works

Server-side validation:

- Employee ID is required and must be unique.
- First and last names cannot be empty.
- Email is required, must match the implemented email pattern, and must be unique.
- Phone is required during creation and cannot be blank during updates; creation also checks a minimum length.
- Department, designation, and joining date are required during creation.
- Salary must be numeric and greater than zero.
- Validation failures return **400 Bad Request** with field-specific error information.

✓ Search and filter works

- The **search** parameter checks Employee ID, first name, last name, full name, email, phone, designation, and department.
- The **department** parameter filters by department, while **status** filters Active/Inactive employees.

code

```
GET /api/employees?search=engineering
GET /api/employees?department=Engineering
GET /api/employees?status=Active
```

✓ Authentication endpoints are implemented

- The login endpoint checks the supplied username and password and returns a demo token and user information for the supported demo accounts.
- Missing credentials return 400; invalid credentials return 401.
- Logout returns a successful logout message.

code

```
POST /api/auth/login
POST /api/auth/logout
```

✓ API endpoints can be demonstrated

The backend provides the following REST-style endpoints:

Operation	Method	Endpoint
Health check	GET	/api/health
Login	POST	/api/auth/login
Logout	POST	/api/auth/logout
Dashboard summary	GET	/api/dashboard/summary
List employees	GET	/api/employees
Get employee	GET	/api/employees/:id
Create employee	POST	/api/employees
Update employee	PUT/PATCH	/api/employees/:id
Delete employee	DELETE	/api/employees/:id

✓ Data model and TypeScript interfaces

The supplied `types.ts` defines the application's main data contracts.

```
code

Employee → id, employee_id, first_name, last_name, email, phone,
department, designation, salary, joining_date, address, status,
created_at?, updated_at?

EmployeeFormData → employee form fields
DashboardSummary → total, active, inactive, departmentSummary, recent
User → id, username, name, role, email
FormErrors → field-level validation messages
ViewTab → dashboard | employees | add | api-guide
```

✓ Architecture

```
code

User
↓
React + TypeScript Frontend (Vite)
↓ HTTP / JSON
Express REST API (server.ts)
↓
JSON file persistence
↓
data/employees.json
```

Architecture explanation:

- The frontend provides the user interface and communicates with the Express API using HTTP/JSON.
- The Express backend performs routing, validation, authentication checks, dashboard calculations, and employee CRUD operations.
- Employee records are read from and written to `data/employees.json`.

✓ Code flow for creating an employee

1. The user enters employee information in the frontend form.
2. The frontend sends a `POST /api/employees` request with JSON data.
3. Express extracts the submitted fields and performs validation.
4. Duplicate Employee ID/email, invalid email, invalid phone, and non-positive salary are rejected.
5. A valid request receives a new internal ID and timestamps, then the employee is appended to the stored array.
6. The updated JSON file is saved and the new employee is returned with `201 Created`.

✓ Code flow for read, update, and delete

- **Read:** load the JSON records, apply optional query filters, and return the result.
- **Update:** locate the employee, validate supplied fields, merge the changes, update the timestamp, save, and return the record.
- **Delete:** locate the employee, remove it from the array, save the file, and return a confirmation response.

✓ Source code and project documentation are organized

- `server.ts` — Express server, authentication, dashboard, employee CRUD, validation, search, filters, and persistence.
 - `types.ts` — Employee, form, dashboard, user, error, and view-tab TypeScript definitions.
 - `src/components/` — frontend component area; `src/services/` — service/API integration area.
 - `App.tsx`, `main.tsx`, `index.css` — core frontend files.
 - `backend/`, `data/`, `postman/` — backend/data/API-testing project areas.
 - `README.md`, `package.json`, `tsconfig.json`, `vite.config.ts` — documentation and configuration.
-

✓ Important implementation details

- Express JSON parsing is enabled and CORS headers allow GET, POST, PUT, PATCH, DELETE, and OPTIONS requests.
 - A health-check endpoint is available at `/api/health`.
 - Development uses Vite middleware; production serves the built `dist/` directory.
 - Employee status is restricted by the TypeScript model to `Active` or `Inactive`.
-

✓ Project reference

Project: **Employee Management System**

Application: <https://mychar-employeeerecord-70.ai.studio/>

Completion Statement

The Employee Management System provides a complete employee-record management workflow through a React/TypeScript frontend and an Express REST backend. The implemented API covers authentication, dashboard statistics, employee creation, listing, single-record retrieval, updating, deletion, validation, and search/filter support. Employee data is persisted in a JSON file, while shared TypeScript interfaces define the expected application data structures. The supplied project structure separates frontend components and services, backend code, data, and Postman resources.

Documentation basis: supplied reference documentation PDF, project structure screenshot, `server.ts`, and `types.ts`. Frontend-specific behavior not directly present in the supplied source files is not presented as independently verified.